



ACADÉMIE
DE NANTES

Liberté
Égalité
Fraternité

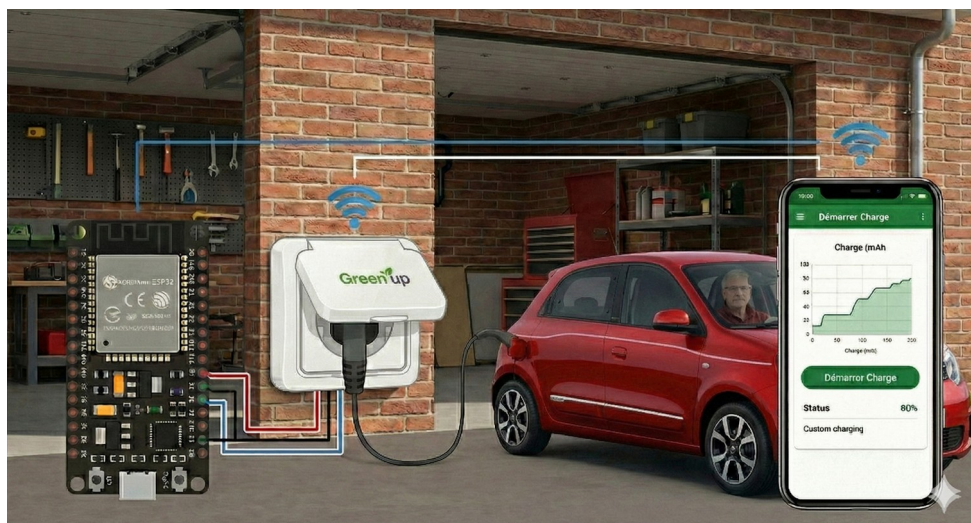


Lycée
**TOUCHARD
WASHINGTON**

Dossier Personnel

BTS CIEL SESSION 2026 – IR E6 – PROJET TECHNIQUE

Gestionnaire de charge de véhicules électrique



Lycée : <u>Touchard-Washington</u>		Ville : LE MANS	
Nom du projet :	Gestionnaire de charge de Véhicules électriques		
N° du projet :	TW2		
Projet nouveau	Oui ☐ Non ☐	Projet interne	Oui ☒ Non ☐
Délai de réalisation	150 heures	Statut des étudiants	Formation initiale ☒ Apprentissage ☐
Spécialité des étudiants	ER ☐ IR ☒ Mixte ☐	Nombre d'étudiants :	4 étudiants
Professeurs responsables	Philippe CRUCHET , Didier Bernard, François Martin, Anthony LE <u>CREN</u> , Jilali <u>KHAMACH</u> , François <u>QUEREC</u>		

Table des matières

1. Cas d'utilisation.....	4
1.1. Mesurer la puissance.....	4
1.2 Notifier en cas d'erreur ou en cas d'alerte.....	5
1.3 Mémoriser les paramètres de la charge.....	6
2. Composants matériels.....	7
2.1 Capteur de Courant.....	7
2.2 Capteur de Tension.....	7
2.3 La RaspberryPi.....	8
2. Protocoles.....	9
2.1. JSON.....	9
2.2 WebSocket.....	10
3. Outils.....	10
3.1 QT Creator.....	10
Sortie Console (Débogage).....	11
WebSockets.....	11
Lecture/Écriture JSON.....	11
Format JSON et Gestion de la Base de Données.....	11
Persistance des données (SQL).....	11
3.2 MariaDB.....	12
3.4 Visual Studio Code.....	13
Coloration et Auto-complétion (Aide au codage).....	13
Gestion des Extensions.....	13
Console et Terminal Intégrés.....	13
4. Présentation de GitHub.....	14
Gestion de version avec GitHub.....	14
5. Diagrammes de séquences.....	15
5.1 Mesurer la Puissance.....	15
5.2 Notifier en cas d'erreur ou en cas d'alerte.....	16
5.3 Mémoriser les paramètres de charge.....	18
6. Diagrammes de classe.....	19
6.1 Diagramme de classe pour l'ESP 32.....	19
6.2 Diagramme de classe de la Raspberry Pi.....	21
7. Fiche de test.....	23
8. Diagramme de Gantt.....	27
9. Compétences acquises.....	28
10. Perspectives d'améliorations.....	29

1.Cas d'utilisation

1.1. Mesurer la puissance

Description du cas D'utilisation	L'ESP32 surveille en permanence les capteurs afin de détecter tout dépassement de seuil critique. En cas d'anomalie, le système coupe immédiatement l'alimentation, enregistre l'erreur en base de données et envoie une alerte en temps réel pour notifier l'utilisateur.
Pré conditions	➤ La session de charge est active. La connexion à la base de données est établie.
Post conditions	➤ Les données de charge sont archivées et consultables. ➤ L'historique est à jour avec une moyenne de la puissance envoyé toutes les 60 secondes.
Scénario principale	L'horloge système envoie un signal de scrutation chaque seconde. • Mesure : Le système interroge les capteurs de courant et de tension. • Calcul : Le système fait une somme des tensions et des intensités • Moyenne : Le système effectue une moyenne de la somme des tensions et intensités par le nombre de lectures prises.
Alternatives	A1 : Perte de connexion réseau (Wi-Fi ou WebSocket) Si l'ESP32 perd la connexion avec la Raspberry Pi au moment d'envoyer le rapport de la minute, la trame JSON n'est pas transmise. Comme l'ESP32 ne possède pas de stockage local, la mesure de cette puissance moyenne est définitivement perdue. A2 : Base de données MariaDB temporairement injoignable Si le serveur Raspberry Pi reçoit correctement la trame de l'ESP32 mais que la connexion à la base de données a échoué, le code du serveur lève une erreur dans le terminal (journalctl), mais la mesure ne peut pas être enregistrée dans la table MESURE.

1.2 Notifier en cas d'erreur ou en cas d'alerte

Description du cas D'utilisation	L'ESP32 surveille en permanence les capteurs afin de détecter tout dépassement de seuil critique. En cas d'anomalie, le système coupe immédiatement l'alimentation, enregistre l'erreur en base de données et envoie une alerte en temps réel pour notifier l'utilisateur.
Pré conditions	➤ Une session de charge est en cours (numéro de session actif)
Post conditions	➤ L'utilisateur est informé de l'arrêt de la charge. ➤ La borne est placée dans un état sécurisé.
Scénario principale	Détection : Le système détecte une anomalie : • Erreur de courant : dépassement du seuil d'intensité (ex: courant supérieur à 32A). • Erreur de température : mesure hors plage de sécurité (ex: température supérieure à 253V). • Identification : Le système récupère le numéro de session de charge actuellement actif en mémoire ou dans la base de données. • Recherche : Le système identifie l'utilisateur ou l'interface associée à ce numéro de session spécifique. • Notification : Le système génère et envoie un message d'alerte. • Journalisation : L'erreur est enregistrée dans l'historique avec l'horodatage et l'ID de session.
Alternatives	

1.3 Mémoriser les paramètres de la charge

Description du cas D'utilisation	L'ESP32 surveille en permanence les capteurs afin de détecter tout dépassement de seuil critique. En cas d'anomalie, le système coupe immédiatement l'alimentation, enregistre l'erreur en base de données et envoie une alerte en temps réel pour notifier l'utilisateur.
Pré conditions	➤ Connexion de la borne à la base de données, une session de charge a été initialisée.
Post conditions	➤ Les paramètres complets de la session sont disponibles en base de données pour l'utilisateur ou l'administrateur.
Scénario principale	• Collecte et envoi : Toutes les 60 secondes, l'ESP32 calcule la puissance moyenne consommée et l'envoie au serveur Raspberry Pi sous forme de trame JSON ({"puissance": valeur}). • Identification : Le serveur réceptionne la trame et l'associe à l'identifiant de la session de charge en cours (id_charge). • Horodatage et stockage : Le serveur exécute la requête d'insertion dans la table MESURE. C'est le système de gestion de base de données (MariaDB) qui applique automatiquement l'horodatage actuel lors de la création de la ligne. • Confirmation : La base de données confirme l'enregistrement réussi de la mesure.
Alternatives	A1 : Échec de connexion base de données : Le système signale que l'envoi de la trame est impossible vers la base de données.

2. Composants matériels

2.1 Capteur de Courant



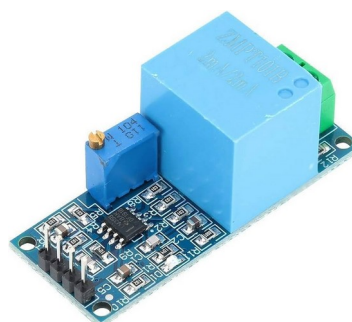
*Illustration 1: Capteur de courant
SCT-018-020*

Le **SCT-013-020**, c'est un capteur de **courant analogique** en forme de pince. On le clipse directement autour du câble d'alimentation de la borne pour surveiller l'intensité de l'électricité pendant que le véhicule recharge.

Il a une plage de mesure qui va de **0 à 20A**, ce qui colle parfaitement à notre projet. Le vrai point fort de ce capteur, c'est son côté "pince" : pas besoin de couper ou de dénuder les câbles électriques pour l'installer, on a juste à le refermer autour du fil en toute sécurité.

Le capteur envoie ses **infos (une tension de 0 à 1V)** sur une broche analogique de l'ESP32. Si le courant dépasse le seuil limite (en cas de surcharge), le gestionnaire de charge réagit direct : il coupe le jus pour sécuriser toute l'installation et envoie immédiatement une alerte.

2.2 Capteur de Tension



*Illustration 2: Capteur de
tension ZMPT101B*

Le ZMPT101B, c'est notre capteur de tension alternative. Tout petit et alimenté initialement en 5V mais réellement en 3,3V pour l'ESP 32, il sert à mesurer en continu le secteur (autour de 230V) pendant que la voiture recharge.

Le vrai point fort de ce module, c'est qu'il adapte le signal pour nous : il l'amplifie et le décale dans le positif. C'est indispensable, parce que l'ESP32 est incapable de lire des tensions négatives et risquerait de griller. En plus, il joue un rôle de barrière de sécurité en isolant totalement la haute tension du reste de notre circuit.

Le capteur renvoie un signal analogique tout propre à l'ESP32. En combinant cette tension avec le courant de la pince SCT-013, notre gestionnaire de charge a tout ce qu'il faut pour calculer la puissance exacte consommée par le véhicule.

2.3 La RaspberryPi

La Raspberry Pi 4 (Modèle B) constitue le cœur matériel de notre infrastructure de recharge. Ce micro-ordinateur, fonctionnant sous l'environnement Linux Debian, assure le rôle de serveur central du projet.



Afin de maximiser la réactivité du système et de prévenir les risques de défaillance, le stockage est confié à un **SSD externe d'environ 230 Go** plutôt qu'à une carte SD standard. Ce choix technique permet de supporter efficacement les opérations d'écriture fréquentes requises par la base de données MariaDB.

Illustration 3: RaspberryPi 4

Caractéristiques techniques du serveur :

- **Modèle** : Raspberry Pi 4 Model B Rev 1.2.
- **Système d'exploitation** : Linux Debian 13.4.
- **Mémoire vive (RAM)** : La plateforme dispose d'un total de 3,7 Go de RAM. Les services du serveur Qt et de la base de données n'utilisant actuellement que 553 Mo, le système conserve plus de 3,2 Go de RAM disponible.

En somme, cette configuration offre les ressources nécessaires pour traiter simultanément les flux WebSockets de l'ESP32, les requêtes de l'application mobile et la persistance des données SQL de manière fluide et stable.

2. Protocoles

2.1. JSON

En ce qui concerne l'envoi des données, nous avons choisi le format **JSON** (*JavaScript Object Notation*), que nous avons estimé le plus approprié pour la communication textuelle entre l'ESP32 et le Raspberry Pi. Pour faciliter l'écriture de la trame et optimiser la bande passante, nous avons réduit le nombre d'informations au strict minimum nécessaire à l'application.

Champs	Nom de la clé	Type	Exemple
Action	action	String	« alerte »
Type	type	Entier	0
Message	message	Chaîne	« Dépassement de seuil »
ID Borne	idBorne	Entier	4

Exemples de messages JSON reçus :

- *Alerte Courant* : {"action": "alerte", "type": 0, "message": "Surcharge de courant !", "idBorne": 2}
- *Alerte Température* : {"action": "alerte", "type": 1, "message": "Surchauffe critique !", "idBorne": 2}

2.2 WebSocket

L'utilisation du protocole WebSockets permet de mettre en place une liaison bidirectionnelle et permanente entre la Raspberry Pi (qui agit comme serveur central), l'ESP32 et l'application mobile (les clients). À l'inverse d'une architecture HTTP standard, cette technologie élimine le besoin d'interroger le serveur en boucle (*polling*) grâce au maintien d'un canal de communication constamment ouvert. Par conséquent, dès qu'une mesure de puissance est calculée ou qu'un événement survient, l'information est poussée instantanément vers l'application et la base de données.

Cette approche technique offre des bénéfices concrets pour notre système :

- **Réactivité de l'affichage et des alertes** : Les variations de puissance moyenne ainsi que les anomalies (surchauffe ou surintensité) sont transmises en temps réel dès leur détection, sans aucune latence.
- **Économie des ressources réseau** : En supprimant les requêtes répétitives et inutiles, on réduit drastiquement l'occupation de la bande passante et la charge de traitement sur la Raspberry Pi et l'application mobile.
- **Dynamisme de l'interface** : L'application mobile profite d'une mise à jour fluide et continue des données de la borne, offrant une réactivité indispensable pour la notification immédiate des alertes à l'utilisateur.



3.Outils

3.1 QT Creator

Qt est un framework de développement logiciel multiplateforme utilisé ici pour concevoir une application de type serveur (sans interface graphique) dédiée au traitement de données. Développé par *The Qt Company*, Qt est particulièrement adapté pour fonctionner sur le système d'exploitation **Raspberry Pi OS**. Il nous permet de développer nativement en **C++** et de gérer la persistance des données dans une base de données **SQL** grâce à ses bibliothèques intégrées. C'est donc avec ce logiciel que le serveur de la RaspberryPI a été codé.

Sortie Console (Débogage)

Qt permet de gérer les sorties textuelles en utilisant la classe `QDebug`, qui facilite l'affichage de messages de diagnostic directement dans la console du serveur. Grâce à cette fonctionnalité, nous pouvons suivre en direct les trames reçues ou envoyées, les connexions des clients, les erreurs de requête et les étapes de traitement des données. C'est un outil de suivi indispensable pour la maintenance du système.

WebSockets

La bibliothèque réseau de Qt est exploitée pour gérer le serveur WebSocket. Ce protocole assure la transmission instantanée et bidirectionnelle des mesures et des alertes vers l'application mobile Android des utilisateurs, ainsi que la réception des commandes de configuration.

Lecture/Écriture JSON

Dans ce projet de borne de recharge, la classe `QJsonObject` (ainsi que `QJsonDocument`) est utilisée pour décoder et structurer l'ensemble des messages échangés. Elle prend en charge :

- **La lecture et l'analyse des trames** reçues depuis l'ESP32 (mesures de puissance brute ou alertes de sécurité de type courant/température) ainsi que les requêtes de gestion de flotte provenant de l'application mobile (ajout, modification ou suppression de véhicules).
- **La construction des notifications** renvoyées en temps réel aux clients connectés lors du déclenchement d'une alerte ou de la mise à jour de la base de données.

Format JSON et Gestion de la Base de Données

Le format **JSON** (*JavaScript Object Notation*) est particulièrement adapté pour structurer les messages échangés dans notre système. Grâce aux classes `QJsonObject` et `QJsonDocument`, l'application serveur de la Raspberry Pi peut facilement décoder les valeurs de puissance ou les alertes reçues, garantissant une communication claire et fiable.

Persistance des données (SQL)

Les classes `QSqlDatabase` et `QSqlQuery` de Qt sont utilisées pour enregistrer les informations dans une base de données locale. Ce stockage permet de conserver un historique complet des événements de la borne de recharge et de gérer le parc de véhicules.

Objectifs de l'enregistrement en base de données :

- **Archiver les mesures** : Enregistrer chaque calcul de puissance moyenne avec son horodatage pour le suivi de la consommation.
- **Historiser les alertes** : Garder une trace des défauts de courant (surintensité) et de température (surchauffe) pour la maintenance de la borne.

- **Gestion de flotte** : Stocker et mettre à jour les informations des véhicules de l'application mobile (noms, identifiants, kilométrages).

Rôle des classes Qt :

- **QSqlDatabase** : Permet de configurer, de sécuriser et d'ouvrir la connexion avec le système de gestion de base de données.
- **QSqlQuery** : Permet d'exécuter les requêtes SQL nécessaires à l'application, telles que l'insertion des puissances (`insérerEnBase`), l'archivage des anomalies (`ajouterEvenement`) ou les opérations de modification et suppression des véhicules.

3.2 MariaDB

MariaDB est un Système de Gestion de Base de Données Relationnel open source, dérivé de MySQL et entièrement compatible avec le langage SQL. Reconnu pour sa stabilité, sa performance et sa légèreté, il convient parfaitement à une utilisation embarquée ou serveur dans des projets de type IoT, comme notre système de borne de recharge.



Dans ce projet, MariaDB est installé sur la Raspberry Pi pour centraliser et historiser l'ensemble des informations du système. Ce stockage structuré rend les données accessibles et exploitables en temps réel par le serveur Qt ainsi que par l'application mobile.

MariaDB est utilisé pour :

- **Stocker durablement les mesures** : Enregistrer les valeurs de puissance moyenne calculées par l'ESP32.
- **Historiser la sécurité de la borne** : Archiver de manière chronologique chaque alerte de surintensité (courant) ou de surchauffe (température) avec un horodatage précis.
- **Gérer le parc automobile** : Conserver les données relatives aux véhicules de l'utilisateur (ID, nom, kilométrage) et appliquer instantanément les demandes d'ajout, de modification ou de suppression.
- **Assurer la persistance et l'indépendance des données** : Garantir la sauvegarde et la sécurité des informations, qui restent consultables et fiables même en cas de redémarrage ou de coupure du serveur Qt.

3.4 Visual Studio Code



Visual Studio Code est un éditeur de code source open source développé par Microsoft. Très léger et rapide, il est particulièrement apprécié des développeurs pour sa polyvalence et son système d'extensions qui permet de l'adapter à n'importe quel langage de programmation.

Dans ce projet, VS Code a été utilisé pour développer le code de l'ESP32

Coloration et Auto-complétion (Aide au codage)

VS Code analyse le code en temps réel pour colorer la syntaxe et suggérer automatiquement les noms des fonctions ou des variables pendant la saisie. Cette fonctionnalité permet de coder beaucoup plus vite et d'éviter les erreurs d'inattention ou de frappe.

Gestion des Extensions

La force de VS Code réside dans son catalogue d'extensions. Nous avons pu y intégrer des outils spécifiques comme PlatformIO, transformant un simple éditeur de texte en un environnement de développement complet et sur mesure.

Console et Terminal Intégrés

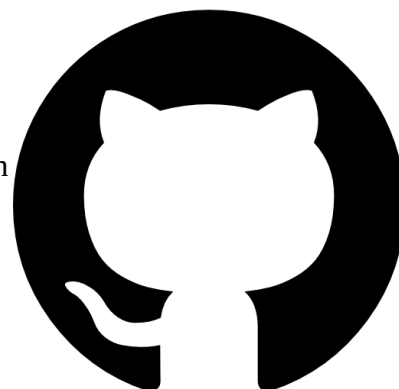
L'intégration d'un terminal directement dans l'interface de VS Code permet de lancer les scripts, de téléverser le code dans les cartes électroniques ou de surveiller les retours de la console (debug) sans jamais avoir à quitter l'application. C'est un gain de temps précieux pour le développement et les tests.

4.Présentation de GitHub

Gestion de version avec GitHub

Pour garantir un suivi rigoureux de notre code source, nous avons fait le choix d'utiliser **GitHub**. Cette plateforme a été le pilier de notre organisation technique, nous permettant de versionner, de centraliser et de sécuriser l'ensemble des briques logicielles de notre borne de recharge (serveur Raspberry Pi, code embarqué de l'ESP32 et application mobile).

L'intégration de cet outil dans notre flux de travail nous a apporté plusieurs bénéfices majeurs :



- **Traçabilité et sécurité du code** : Chaque évolution du programme est sauvegardée et documentée (les *commits*). Si un bug apparaît lors d'un test matériel avec la borne, cette historisation nous permet de restaurer instantanément une version précédente et stable du code.
- **Travail d'équipe optimisé** : GitHub excelle dans la gestion du travail collaboratif. Le système de branches permet à chaque membre de développer une fonctionnalité de son côté (par exemple, l'interface mobile ou la communication WebSocket) sans risquer de casser le code des autres, avant de fusionner proprement le travail.
- **Centralisation cloud** : L'intégralité du projet étant hébergée sur des serveurs distants, nous sommes totalement prémunis contre les pannes matérielles locales (perte d'un ordinateur personnel ou corruption de la carte SD de la Raspberry Pi).
- **Démarche professionnelle** : En tant que futurs Concepteurs et Développeurs d'Application, la maîtrise d'un outil de versioning comme Git est une compétence incontournable. L'utiliser tout au long de ce projet nous a permis d'adopter de solides standards de l'industrie logicielle.

En définitive, l'adoption de GitHub a grandement fluidifié notre cycle de développement en structurant nos échanges. Cela a été un atout déterminant dans la réussite technique de notre infrastructure de recharge connectée.

Et voici le lien afin d'accéder au GitHub du groupe :

https://github.com/Corentinh2/PROJET_GESTIONNAIRE_DE_CHARGE

5. Diagrammes de séquences

Après avoir détaillé les cas d'utilisation et justifié le choix de notre environnement technique, nous pouvons désormais passer à la modélisation dynamique. Les diagrammes de séquence suivants illustrent avec précision les interactions réelles entre les différents sous-systèmes (ESP32, Raspberry Pi et application mobile).

5.1 Mesurer la Puissance

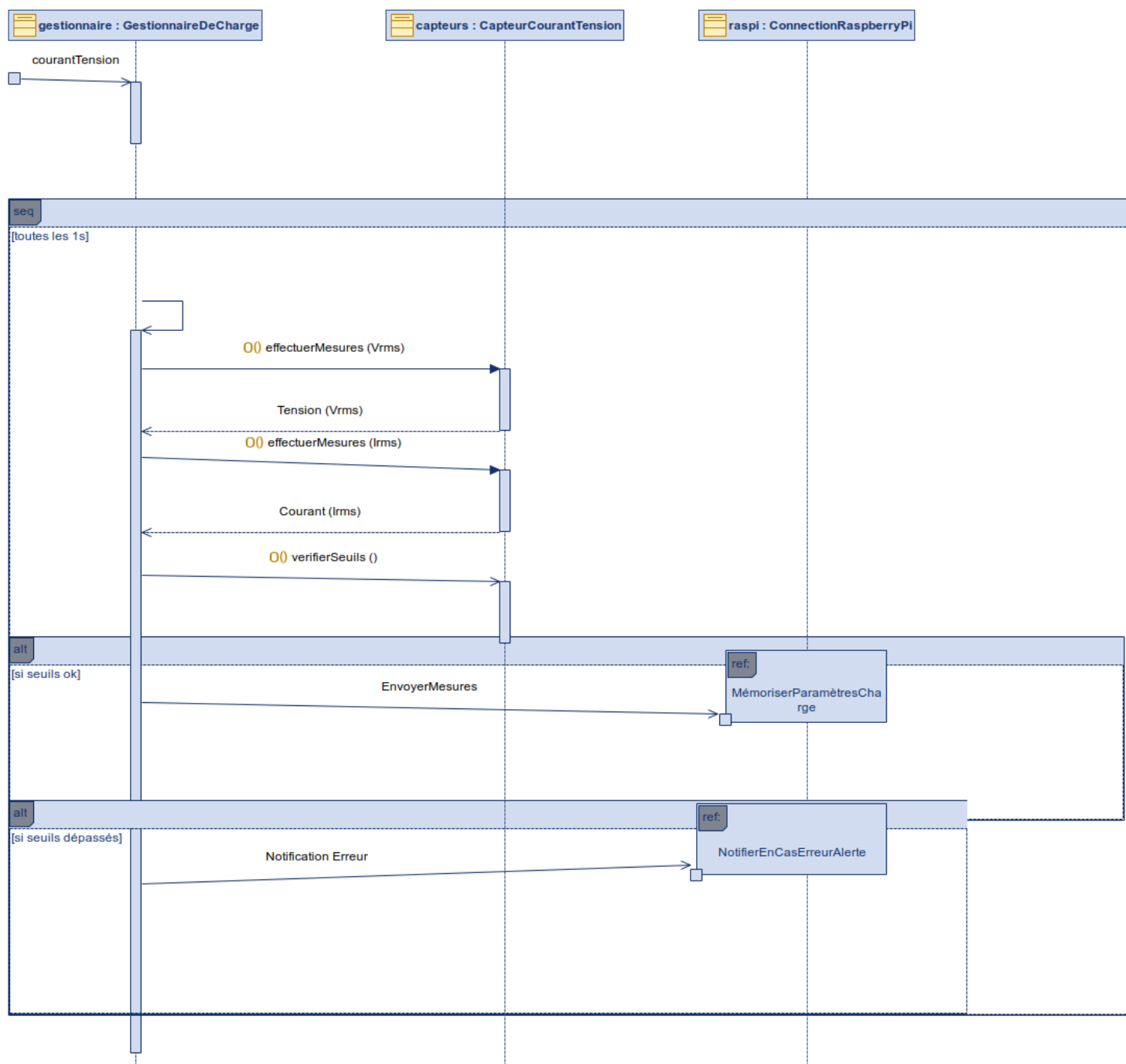


Illustration 4: Diagramme de séquence "Mesurer la Puissance"

5.2 Notifier en cas d'erreur ou en cas d'alerte

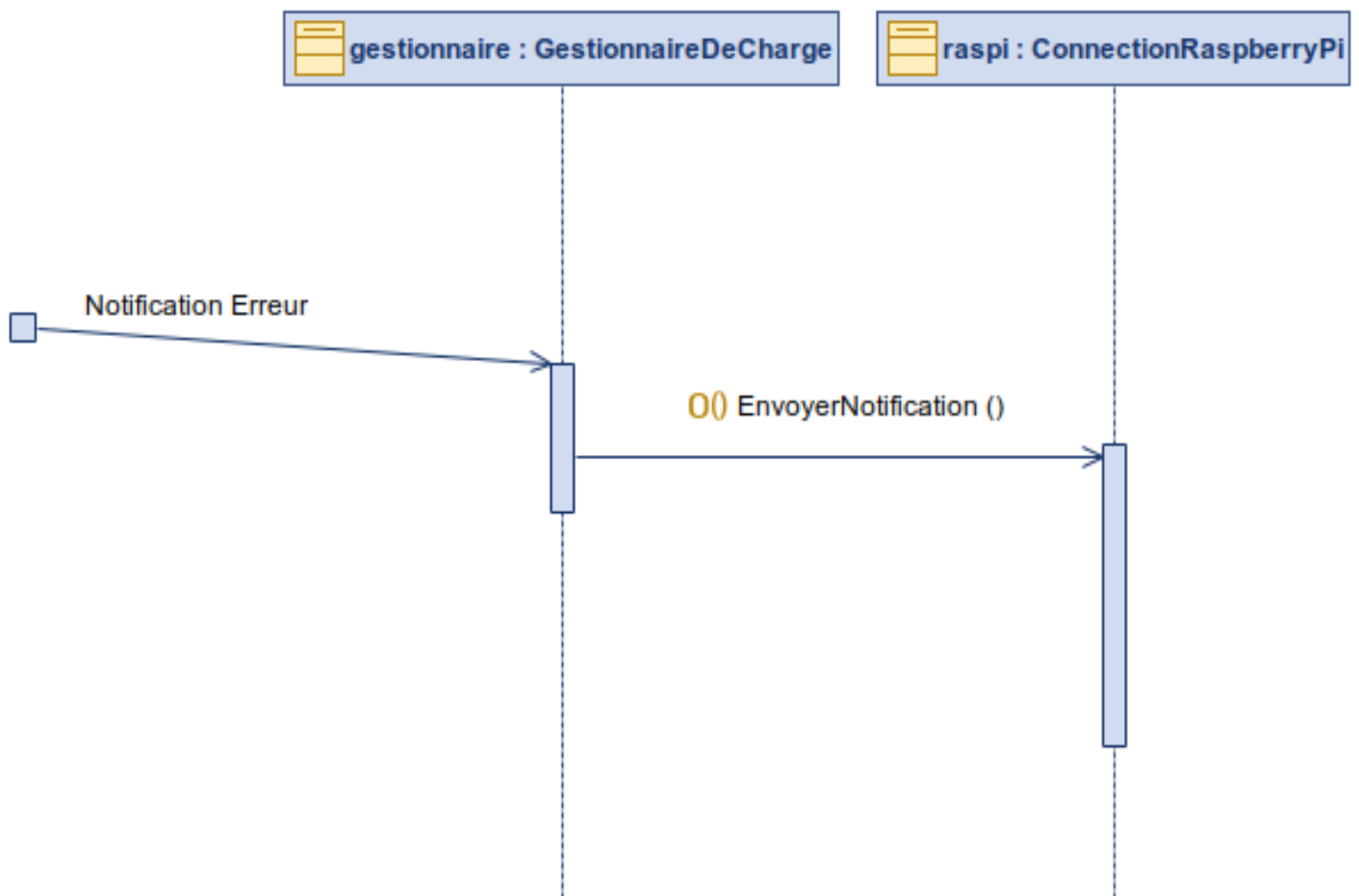


Illustration 5: Diagramme de séquence "Notifier en cas d'erreur ou en cas d'alerte"

5.3 Mémoriser les paramètres de charge

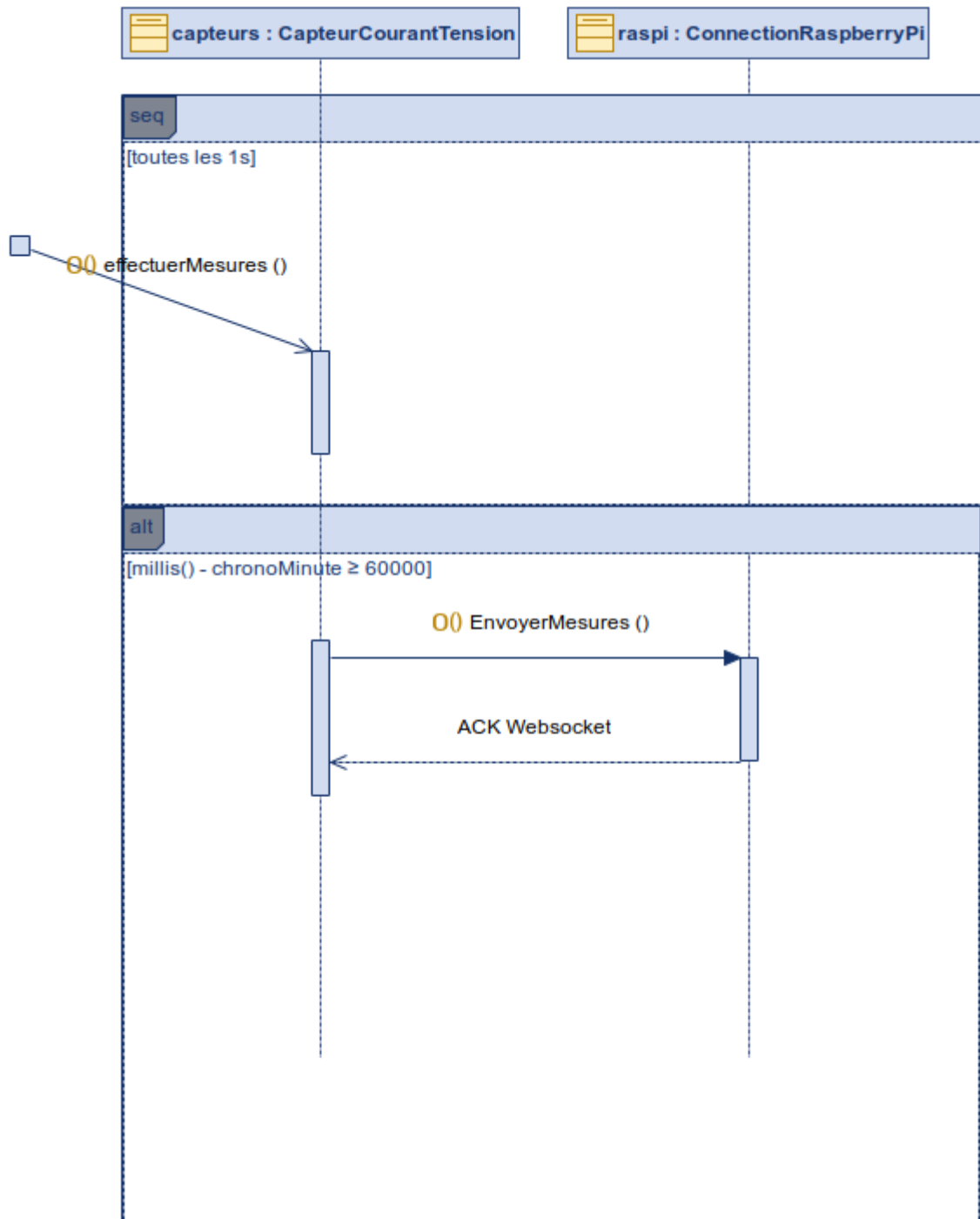


Illustration 6: Diagramme de séquence "Mémoriser les paramètres de charges"

6. Diagrammes de classe

6.1 Diagramme de classe pour l'ESP 32

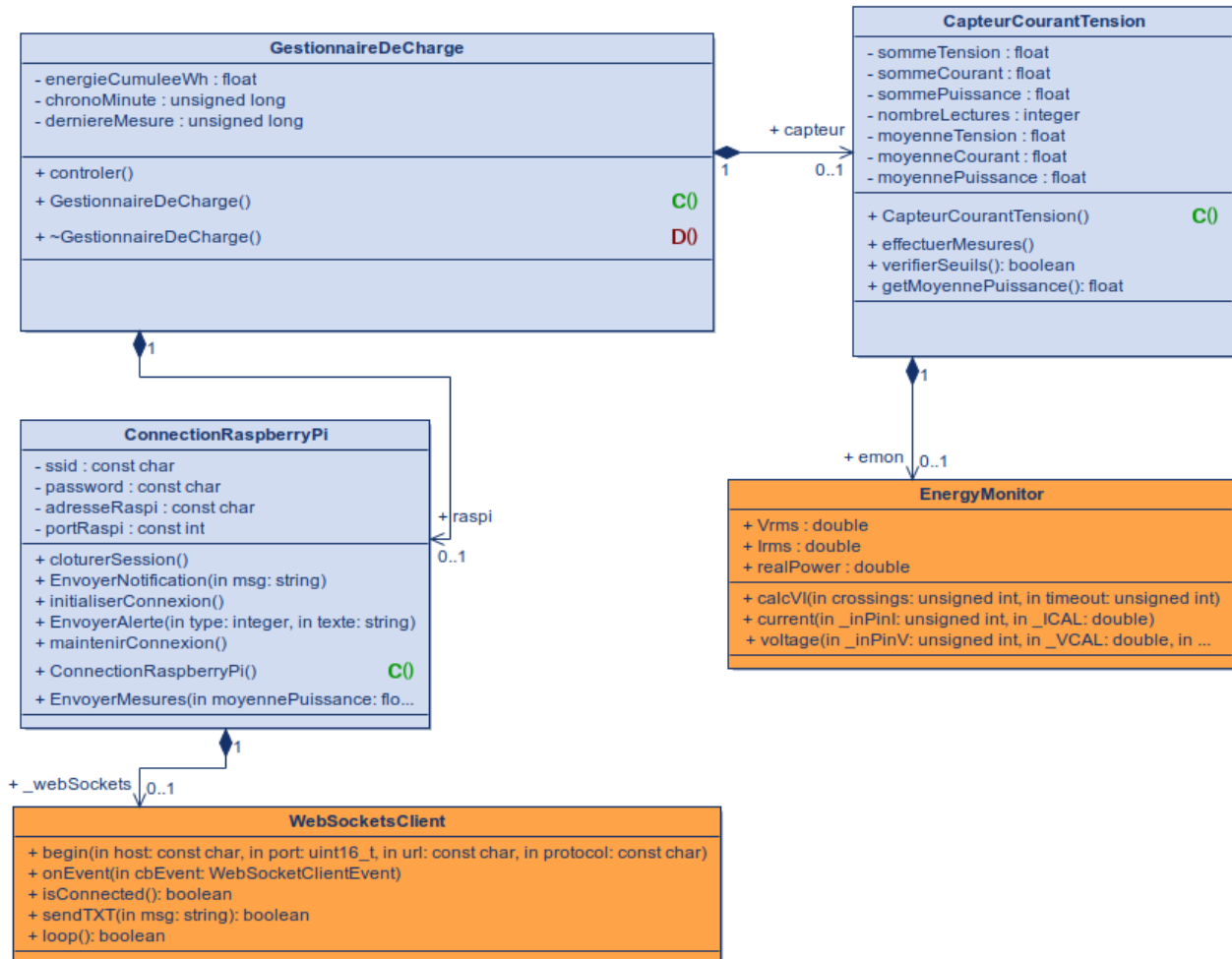


Illustration 7: Diagramme de classe de l'ESP 32

Suite à la définition de notre architecture matérielle et au choix de nos bibliothèques, nous pouvons désormais détailler notre diagramme de classes afin de l'adapter aux contraintes du développement embarqué sur l'ESP32.

Dans ce diagramme, nous avons intégré les classes issues de bibliothèques externes (représentées en orange) qui viennent soutenir nos propres classes de gestion (en bleu) :

- **EnergyMonitor (issue de la bibliothèque EmonLib)** : Permet d'interfacer le programme avec nos capteurs physiques. Elle se charge d'effectuer les calculs mathématiques complexes

sur le signal alternatif pour en extraire les valeurs efficaces (V_{rms} , I_{rms}) et la puissance réelle ($realPower$).

- **WebSocketsClient** : Permet de gérer la couche réseau en établissant une connexion bidirectionnelle et persistante avec le serveur Raspberry Pi. Elle nous offre les méthodes nécessaires pour envoyer nos trames de données sous forme de texte (`sendTXT`).

Afin de garder un code propre et modulaire, ces outils externes sont encapsulés dans nos propres classes métiers :

- **CapteurCourantTension** : Pilote la classe `EnergyMonitor` pour réaliser les lectures, calculer les moyennes lissées des tensions/courants et vérifier les dépassements de seuils critiques.
- **ConnectionRaspberryPi** : Encapsule la classe `WebSocketsClient` pour gérer l'initialisation du Wi-Fi et formater l'envoi de nos messages spécifiques (les mesures de puissance et les alertes).
- **GestionnaireDeCharge** : Joue le rôle de contrôleur principal. Elle orchestre les mesures, gère les chronomètres et calcule l'énergie cumulée en Wh en faisant le lien entre les capteurs et la communication réseau.

6.2 Diagramme de classe de la Raspberry Pi

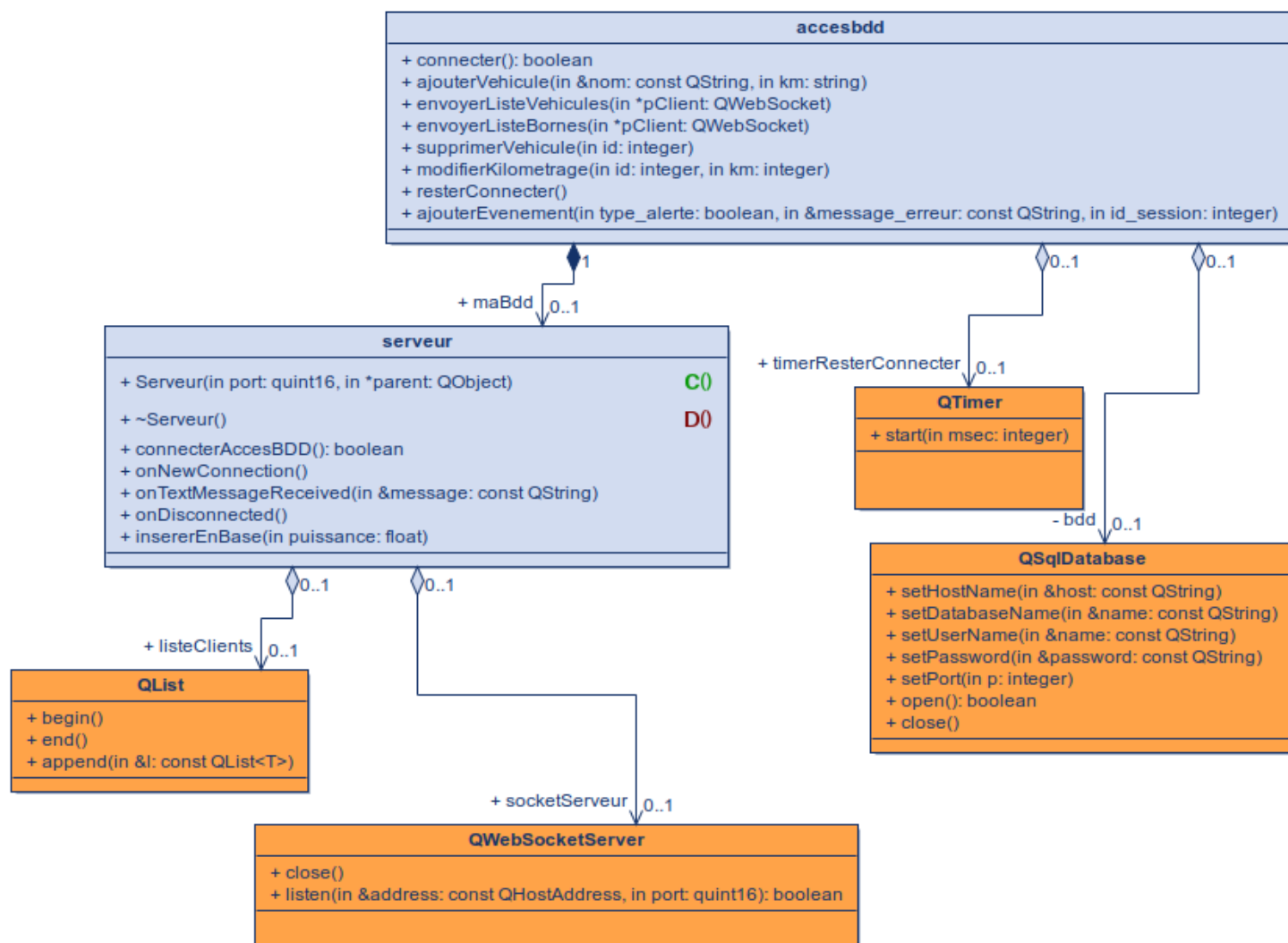


Illustration 8: Diagramme de classe de la Raspberry Pi

Comme pour le précédent, j'ai structuré le texte en séparant tes classes métiers des classes fournies par l'environnement de développement.

De la même manière que pour le microcontrôleur ESP32, nous avons modélisé l'architecture logicielle de notre serveur Raspberry Pi. Ce diagramme met en évidence l'intégration des classes

natives du framework Qt (en orange) au sein de nos propres classes de gestion (en bleu) afin d'assurer la communication réseau et la persistance des données.

Voici le détail des classes fournies par Qt Creator et intégrées à notre projet :

- **QWebSocketServer** : Gère la création du serveur et l'écoute sur un port spécifique pour accepter les connexions entrantes des différents clients (l'ESP32 et l'application mobile).
- **QList** : Fournit une structure de données dynamique permettant de stocker, lister et gérer l'ensemble des clients actuellement connectés au serveur (`listeClients`).
- **QSqlDatabase** : Gère la configuration (hôte, nom d'utilisateur, mot de passe) et l'ouverture de la connexion avec notre base de données MariaDB.
- **QTimer** : Permet d'exécuter des fonctions à intervalles réguliers. Dans notre cas, il est utilisé pour envoyer des requêtes périodiques afin de maintenir la connexion à la base de données active (`resterConnecter`).

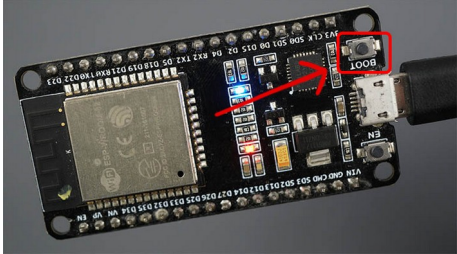
Pour conserver une architecture logicielle claire et modulaire, ces outils sont pilotés par nos deux classes métiers principales :

- **serveur** : C'est le cœur de notre application réseau. Cette classe initialise le serveur WebSocket et gère les événements de connexion/déconnexion. Surtout, elle réceptionne et décode toutes les trames JSON entrantes (`onTextMessageReceived`) avant d'orienter l'action à réaliser (enregistrement d'une puissance, gestion d'une alerte, etc.).
- **accesbdd** : Cette classe encapsule toute la logique d'interaction avec la base de données MariaDB. Elle fait le lien entre les demandes du **serveur** et la base pour exécuter les requêtes SQL complexes, que ce soit pour l'historique des événements (`ajouterEvenement`) ou la gestion de la flotte automobile de l'utilisateur (`ajouterVehicule`, `modifierKilometrage`, `supprimerVehicule`).

7. Fiche de test

<u>Référence du module testé :</u>		FN°ETu.N°Test
Date du test :	30 avril 2026	
Type du test :	Test fonctionnel	
Objectif du test :	Valider que la classe <code>GestionnaireDeCharge</code> est capable de : 1. Calculer et afficher la consommation électrique en temps réel. 2. Déclencher une alerte de sécurité uniquement en cas de dépassement de seuil. 3. Envoyer les mesures sur le serveur de la Raspberry Pi.	
<u>Conditions du test</u>		
État initial Matériel (Hardware) : - Une carte ESP32 (Lolin32) reliée à l'ordinateur par câble USB. - Une Raspberry Pi allumée, connectée au même réseau WiFi que l'ESP32. - Un capteur de tension branché sur la broche 32 et un capteur de courant sur la broche 33. Logiciel (Software) : - Le Moniteur Série de VS Code (ou Arduino IDE) ouvert avec une vitesse de	Environnement du test Côté ESP32 : - Le code est téléversé avec succès sur la carte. - Dans le fichier <code>main.cpp</code>, le bloc de simulation de surcharge (25A) est initialement commenté pour commencer par un test de mesures normales. Côté Réseau : - La Raspberry Pi a une adresse IP fixe connue par l'ESP32. - La liaison WiFi est stable et l'objet	

115200 baud.		
- Le Serveur Qt (service <code>gestionnaire_charge.service</code>) actif sur la Raspberry Pi pour recevoir les données. - Un accès à la base de données « gestionnaire_de_charge » pour vérifier l'enregistrement final. - L'id_charge saisi dans « <code>insererEnBase</code> » correspond à une session de charge existante dans la base de donnée.	<code>raspi</code> est prêt à initialiser la connexion WebSocket. Côté Base de Données : - La table BORNE contient au moins une ligne avec l'ID 1, pour permettre l'enregistrement des alertes sans erreur de clé étrangère. - La table EVENEMENT est consultable - pour vérifier l'arrivée des données.	
Procédures du test		
N°	Opérations	Résultats attendus
1	Connection au serveur Raspi à distance par ssh : ouvrir un terminal sur l'ordinateur, vérifier que la raspi est brancher en ethernet au réseau local. Puis se connecter avec le mdp : « <code>raspberry</code> ».	<code>vruelle@b108tu3p2:~\$ ssh pi@172.18.59.133</code> Résultat : Le terminal affiche <code>pi@raspberrypi:~ \$</code>
2	Surveillance du serveur : Lance la lecture des messages envoyés par ton programme Qt.	Taper : <code>journalctl -u gestionnaire_charge.service -f</code> Résultat : <code>[2159]: Connecté à GestionnaireDeCharge ! [2159]: SERVEUR VERSION 2.0 - TEST 11h42 8080 [2159]: VERSION FINALE TEST maj 09/04</code>
4	Connexion de l'ESP32 à la Raspi : Ouvrir le projet situé : <code>/home/USERS/ELEVES/CIEL2024/vruelle/Documents/Projet_GIT/Code/ProjetGestionnaireDeCharge</code> . Puis le compiler.	Résultat sur VS Code : <code>--- DEMARRAGE DU SYSTEME --- Connexion au Wi-Fi. Connecté ! Adresse IP de l'ESP32 : 192.168.2.91</code> Résultat sur le terminal de la Raspi : <code>[CLIENT CONNECTÉ] - ">::ffff:192.168.2.91"</code>

4.1	Si les messages convenu ne s'affiche pas c'est qu'il faut re compiler le programme directement à partir de l'ESP 32 car il garde en mémoire l'ancien programme (code embarqué) 							
5	Détermination du « point zéro » : Avant de mettre en charge le véhicule en le branchant , il faut attendre que notre ESP 32 initialise sont point à zéro pour garder des valeurs de mesures cohérentes avec la réalité	<pre>Point zero detecte : 1856</pre>						
6	Prise des mesures : toutes les secondes les capteurs prennent les mesures qui sont lus par l'ESP 32. Avec ces données on calcul la puissance qu'on affiche sous forme de tableau	<table border="1"> <thead> <tr> <th>Tension (V)</th> <th>Courant (A)</th> <th>Puissance (W)</th> </tr> </thead> <tbody> <tr> <td>0.0 V</td> <td>1.05 A</td> <td>0.0 W</td> </tr> </tbody> </table>	Tension (V)	Courant (A)	Puissance (W)	0.0 V	1.05 A	0.0 W
Tension (V)	Courant (A)	Puissance (W)						
0.0 V	1.05 A	0.0 W						
7	Moyenne de la minute : toutes les 60 secondes, une moyenne est établie avec toutes les mesures prise la dernière minute.	<pre>--- RAPPORT DE LA DERNIERE MINUTE --- U Moy (V) I Moy (A) P Moy (W) Conso (Wh) 0.0 1.10 0.0 0.000 Wh</pre>						
8	Envoi de la trame (mesure) : à chaque moyenne établie, celle-ci est envoyé au serveur raspi dans le tableau « MESURE ».	<pre>[WS] Envoi trame : {"puissance":0.00} [WS] Message envoyé : {"puissance":0.00}</pre>						
9	Vérification de l'introduction de la mesure : On vérifie que la mesure s'est bien introduite dans le tableau « MESURE », tout d'abord en regardant dans la fenêtre destiné au serveur (terminal de la raspi), puis en réalisant cette commande dans la fenêtre de la base de donnée « gestionnaire_de_charge ». Commande : « SELECT * FROM MESURES ; »	Résultat dans le terminal de la raspi : <pre>[2159]: Trame reçue : "{\"puissance\":0.00}" [2159]: Mesure insérée ! Puissance: 0 W</pre> Résultat dans la bdd « gestionnaire de charge » : <pre>MariaDB [gestionnaire_de_charge]> select * from MESURE; +-----+-----+-----+-----+ id_mesure horodatage puissance id_charge +-----+-----+-----+-----+ 91 2026-04-09 11:52:15 0 4 +-----+-----+-----+-----+</pre>						

10	Test de l'alerte courant : pour cela on va simuler une sur tension. On enlève des commentaires la partie de programme dans le void loop situé dans le main.cpp. <pre>/*static unsigned long chronoTest = 0; if (millis() - chronoTest > 10000) { Serial.println("\n[TEST] Simulation d'une surcharge à 25A..."); systeme.verifierSeuils(25.0, 230.0); chronoTest = millis(); }*/</pre>	<pre>static unsigned long chronoTest = 0; if (millis() - chronoTest > 10000) { Serial.println("\n[TEST] Simulation d'une surcharge à 25A..."); systeme.verifierSeuils(25.0, 230.0); chronoTest = millis(); }</pre>
12	Upload le programme Visual Studio Code qui est désormais décommenté (Ctrl + Alt + U).	
11	Notification de l'alerte : lors d'une sur tension une notification apparaît dans le terminal de notre VS Code	<pre>[TEST] Simulation d'une surcharge à 25A... !!! ALERTE COURANT : 25.00 A !!!</pre>
12	Envoi de l'alerte dans la table « EVENEMENT » : Lors d'une sur tension et donc d'une alerte, on envoie la trame de celle ci dans la bdd, avec l'action, le type, le message .	<pre>[WS] Message envoyé : [WS] Message reçu : { {"action":"alerte","type":0,"message":"Surcharge detectee : 25.00A","idBorne":1} {"action":"alerteRecue","message":"Surcharge detectee : 25.00A","type":0}</pre>
13	Vérification de la réception de la trame notification : Comme pour les mesures, on vérifie dans le terminal de la raspi, puis dans la table « EVENEMENT » de la bdd. <u>Commande SQL</u> : « SELECT * FROM EVENEMENT ; »	Dans le terminal de la raspi : <pre>: Alerte COURANT enregistrée. : Trame reçue : "{\"action\":\"alerte\", \"type\":0, \"me : Succès : Événement enregistré -> Type: \"COURANT (0)\"</pre> Dans la table « EVENEMENT » : <pre>MariaDB [gestionnaire_de_charge]> select * from EVENEMENT; +-----+-----+-----+-----+-----+ id_event type_alerte message_erreur id_borne horodatage +-----+-----+-----+-----+-----+ 1 1 test 1 2026-04-10 12:05:38 2 0 Surcharge detectee : 25.00A 1 2026-04-10 14:36:38 +-----+-----+-----+-----+-----+</pre>

Le test fonctionnel réalisé sur la classe GestionnaireDeCharge a été effectué avec succès. Les différents cas d'usage ont été validés, et les données (mesures et alertes) ont été retrouvées conformes dans les tables concernées de la base de données, sans erreur ni comportement inattendu.

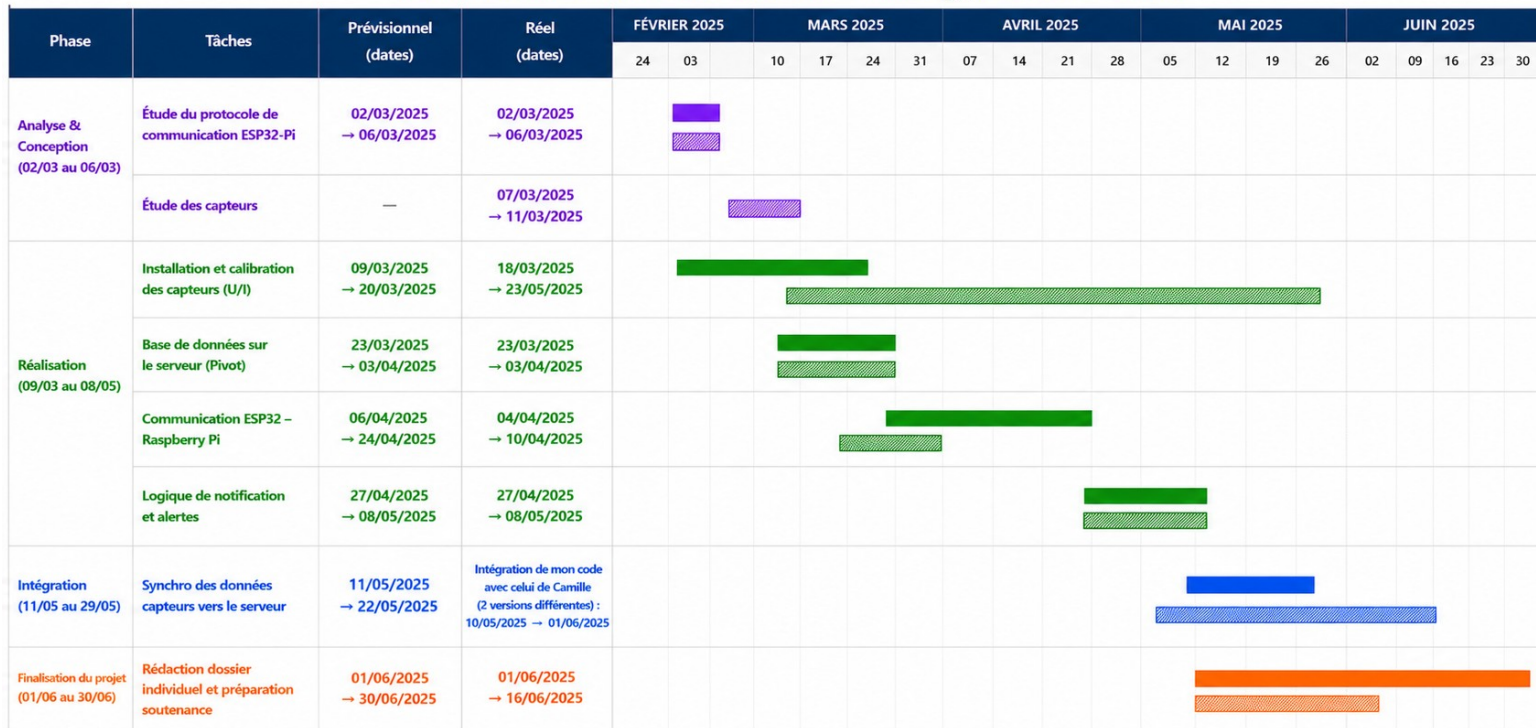
Aucun problème n'a été constaté :

- La **connexion réseau via WebSocket** entre la carte ESP32 et le serveur de la Raspberry Pi a été établie et maintenue correctement.
- Le **calcul de la consommation électrique** en temps réel et l'envoi de la moyenne par minute ont produit les résultats attendus.
- Le **déclenchement de l'alerte de sécurité** a parfaitement fonctionné lors de la simulation d'une surcharge à 25A.

- L'intégrité des données dans les tables MESURES et EVENEMENT (notamment le respect des clés étrangères liées aux ID de charge et de borne) a été respectée lors de l'insertion.

8. Diagramme de Gantt

Étudiant 2 : Mesures & Communication – Planning prévisionnel vs Réel



■ Prévisionnel ■ Réel
Les dates sont incluses (du jour de début au jour de fin).

Illustration 9: Diagramme de Gantt Prévisionnel et Réel

La comparaison entre le diagramme de Gantt prévisionnel et le planning réel montre que le calendrier a dû s'adapter aux réalités techniques du développement.

Dès le démarrage, l'Étude des capteurs, suivie de difficultés lors de l'« Installation et calibration des capteurs (U/I) », a entraîné un allongement important de cette phase. Cependant, ce retard a été immédiatement compensé par la partie « Communication ESP32 – Raspberry Pi », finalisée de manière beaucoup plus rapide que prévu en seulement quelques jours.

Malgré une phase d'intégration plus longue pour fusionner deux versions différentes du code avec Camille, l'avance accumulée sur les autres développements a permis de sécuriser la fin de projet. Cela a permis de finaliser la rédaction du dossier individuel et la préparation de la soutenance à temps.

9. Compétences acquises

Ce projet a été l'opportunité idéale de renforcer mes compétences techniques et professionnelles. Il m'a permis d'adapter mes choix de conception et de développement d'applications aux réalités et aux exigences actuelles du secteur.

En effet, le projet m'a apporté plusieurs choses :

- D'interagir avec différents composants matériels tels qu'un **capteur de courant et un capteur de tension**.
- De **découvrir et d'interagir** avec des bibliothèques et des extensions intégrer à Visual Studio Code.
- **Mettre en œuvre le protocole WebSocket** afin d'établir une communication bidirectionnelle et en temps réel entre l'ESP32 et l'application mobile.
- **Exploiter le format JSON via la bibliothèque ArduinoJson** pour structurer, sérialiser et analyser (parser) efficacement les trames de données échangées.
- D'approfondir mes connaissances sur l'**environnement embarqué**.

Mais aussi sur un plan plus professionnel, le projet m'a permis de découvrir d'autres aspects :

- L'**organisation** entière et global d'un projet, en comprenant le diagramme de Gantt, les push GitHub ainsi que la répartition des tâches de chacun.
- Faire face à des **contraintes et des exigences** dû au matériel utilisé.
- Développer un **esprit critique** sur les problèmes rencontrés.
- **Être objectif** face à des décisions prises.

10. Perspectives d'améliorations

Bien que les cas d'utilisation implémentés répondent aux exigences initiales de la borne de recharge, l'évaluation du prototype révèle des opportunités d'amélioration pour améliorer le comportement global de l'application :

- **Auto-étalonnage adaptatif des capteurs** : Les puces ESP32 possèdent des variations de fabrication qui rendent leurs lectures analogiques imprécises d'un composant à l'autre. Actuellement, le programme utilise des coefficients de calcul fixes. L'ajout d'un mode "calibration" permettant de comparer les mesures de l'ESP32 avec les valeurs réelles d'un multimètre permettrait d'ajuster automatiquement ces coefficients. Cela rendrait le code universel et transposable sur n'importe quel autre ESP32 sans recalibrage manuel.
- **Stockage local temporaire (Mode hors-ligne)** : Pour éviter la perte définitive des données en cas de coupure Wi-Fi ou de redémarrage de la Raspberry Pi, on pourrait utiliser la mémoire flash interne de l'ESP32 (via LittleFS). Les mesures y seraient stockées temporairement, puis envoyées en bloc dès que la connexion est rétablie.
- **Historisation éco-énergétique (Agrégation des données)** : Avec un relevé toutes les 60 secondes, la base de données MariaDB va grossir très rapidement. Une optimisation consisterait à créer un script sur la Raspberry Pi qui calcule et stocke une moyenne horaire ou quotidienne pour les anciennes sessions, puis supprime les lignes de détails (à la minute) de plus d'un mois.